코드북

이 문서는 포트폴리오 저장소의 화면 코드와 빌드 스크립트를 파일 단위로 읽는다. 함수 목록과 유사코드는 `docs/code-reference.pdf`에 있고, 요구사항부터 시험 명세까지는 `docs/technical-specs.pdf`에 있다. 여기서는 그 목록을 다시 찍지 않고, 파일을 위에서 아래로 따라가며 무엇이 바뀌는지 적는다.

서버 프로세스는 없다. 브라우저는 HTML, CSS, JS를 읽고, 학습 기록 데이터는 파이썬 스크립트가 미리 만든다. `data/` 안의 과제 프로그램 소스는 이 코드북의 대상이 아니다. 빌드가 그 파일을 읽어 요약하는 코드만 다룬다. 학번과 이름은 문서에 적지 않는다.

읽는 순서

1. 메인 페이지와 공통 헤더
2. 이력서 기간
3. 자기소개 페이지
4. 온라인 명함
5. 학습 기록 대시보드
6. 히어로 갤러리
7. 요약 빌드
8. 정적 사이트 데이터
9. 문서 생성 스크립트
10. 리소스 보호 및 보안 가드

한 장을 읽는 법

각 장은 같은 순서를 따른다.

- 이 파일이 맡는 일
- 읽는 순서
- 구간 설명
- 이 파일이 부르는 다른 파일

인용은 갈래를 이해하는 데 필요한 몇 줄만 둔다.

메인 페이지와 공통 헤더

이 파일이 맡는 일

`index.html`은 지원 동기, 성장·학습, 역량, 자료, 강점, 마치며, 참고 절을 담은 정적 페이지다. 동작은 `js/main.js`가 맡는다. 즉시 실행 함수 하나가 내비게이션, 자료 드롭다운, 테마를 연결한다. 상단 막대의 배치와 색은 `css/site-header.css`가, 본문 절의 배치는 `css/styles.css`가 맡는다.

`index.html`의 읽는 조각은 이렇다.

- `header.site-header`: 브랜드, `nav#site-nav`, 테마 버튼
- 절 링크: `#section-motivation`, `#section-growth`, `#section-skills`, `#section-strengths`, `#section-closing`, `#section-refs`
- 자료 드롭다운: 같은 페이지의 `#section-links`, 그리고 자기소개, 명함, 이력서, 학습 기록으로 나가는 링크
- `main#main`: 히어로 문구와 본문 절

`css/site-header.css`는 `html[data-theme="light"]`일 때의 색, 링크와 햄버거 버튼의 포커스, 768px 이하에서 메뉴를 접는 배치를 정한다. 769px 이상에서는 메뉴가 한 줄로 펼쳐진다. `css/styles.css`는 메인 본문의 히어로, 절, 카드 간격을 정하고 테마 변수에 맞춰 글자색을 바꾼다.

읽는 순서

파일은 저장 키와 도우미부터 선언하고, 맨 아래에서 DOM이 준비되면 초기화 함수 넷을 부른다.

1. `prefersReducedMotion`, `queryAll`, `queryAllIn`
2. `initSmoothNav`, `initResourceDropdown`, `initThemeToggle`
3. `applyTheme`, `getStoredTheme`, `setStoredTheme`
4. `setResourceDropdown`, `closeResourceDropdowns`, `closeMobileNav`, `setMobileNav`, `debounce`
5. `initSectionTargets`
6. `DOMContentLoaded` 또는 이미 준비된 문서에서 초기화

테마 저장 키는 `portfolio-theme-v2`다. 다른 페이지의 테마 스크립트도 같은 키를 쓴다.

구간 설명

prefersReducedMotion

인자는 없다. `matchMedia("(prefers-reduced-motion: reduce)")`가 있으면 그 `matches`를 돌려주고, 없으면 거짓이다. 참이면 절 이동 스크롤이 `auto`, 거짓이면 `smooth`다.

queryAll, queryAllIn

queryAll은 문서 전체에서 선택자와 맞는 요소를 배열로 돌려준다. queryAllIn은 같은 일을 한 요소 안에서만 한다. NodeList를 배열로 바꾸어 forEach를 쓰기 위한 도우미다.

initSmoothNav

#site-nav와 .nav-toggle을 찾는다. .site-nav__link 중 href가 #으로 시작하고 길이가 2 이상인 것만 클릭을 가로챈다. 대상 id가 없으면 그 클릭은 무시하고 브라우저 기본 이동을 막지 않는다. 대상이 있으면 스크롤하고 closeMobileNav를 부른 뒤, 그 절에 preventScroll로 포커스를 준다.

햄버거 버튼이 있으면 is-open의 반대로 setMobileNav를 부른다. Escape는 메뉴가 열려 있을 때만 닫고 버튼으로 포커스를 돌린다. resize는 debounce 150밀리초 뒤에, 너비가 768px보다 크면 메뉴를 닫는다.

```
if (window.innerWidth > 768 && nav && toggle) {
  closeMobileNav(nav, toggle);
}
```

initResourceDropdown

각 .site-nav__dropdown의 토글 버튼을 연결한다. 클릭하면 다른 드롭다운을 닫고, 이 버튼의 aria-expanded가 true가 아니면 연다. 이미 열려 있으면 닫는다.

드롭다운 링크가 페이지 안 # 링크이고 대상 절이 있으면 그 절로 스크롤하고 포커스를 준다. 다른 페이지로 가는 링크는 기본 이동을 막지 않는다. 어느 쪽이든 이어서 드롭다운과 모바일 메뉴를 닫는다.

문서 클릭이 드롭다운 밖이면 모두 닫는다. Escape도 드롭다운을 모두 닫는다. 이 Escape 처리와 initSmoothNav의 Escape는 따로 등록된다. 메뉴가 열려 있으면 모바일 메뉴 쪽도 닫힌다.

initThemeToggle

[data-theme-toggle]이 없으면 바로 돌아간다. 저장값이 있으면 그것을, 없으면 html의 data-default-theme를, 그것도 없으면 dark를 쓴다. applyTheme로 화면에 반영한 뒤 버튼 클릭과 storage 이벤트를 연결한다.

클릭은 현재 data-theme가 light면 dark로, 아니면 light로 뒤집는다. setStoredTheme 다음에 applyTheme를 부른다. 다른 탭이 같은 키를 바꾸면 storage가 그 값으로 맞춘다. 새 값이 비어 있으면 다시 기본 테마나 dark로 간다.

applyTheme

인자 theme은 light만 라이트로 두고 나머지는 dark로 정규화한다. 되돌리는 값은 없다. html의 data-theme를 바꾸고, 버튼의 aria-pressed는 다크일 때 true다. aria-label은 지금 테마의 반대로 전환한다고 안내하고, [data-theme-label] 문구는 다크 또는 라이트다.

getStoredTheme, setStoredTheme

getStoredTheme는 localStorage.getItem 결과를 돌려준다. 읽기가 실패하면 null이다. setStoredTheme는 같은 키에 쓴다. 쓰기가 실패하면 예외를 삼키므로 메모리와 화면만 바뀌고 다음 방문에는 기본값으로 돌아간다.

setResourceDropdown, closeResourceDropdowns

setResourceDropdown(dropdown, open)은 is-open 클래스, 버튼의 aria-expanded, 메뉴의 hidden을 같이 맞춘다. closeResourceDropdowns(except)는 except와 같은 요소만 남기고 나머지를 닫는다. except를 생략하면 전부 닫는다.

closeMobileNav, setMobileNav

closeMobileNav는 setMobileNav(..., false)다. setMobileNav는 nav나 toggle이 없으면 아무 것도 하지 않는다. 열림이면 is-open과 메뉴 닫기, 닫힘이면 클래스 제거와 메뉴 열기다. 닫을 때만 closeResourceDropdowns를 부른다.

debounce

debounce(fn, wait)는 타이머를 저장하는 함수를 돌려준다. 다시 호출되면 이전 타이머를 지우고 wait 뒤에 fn을 한 번 실행한다. 창 크기 변경에서 메뉴를 반복해서 닫지 않으려고 쓴다.

initSectionTargets

id가 있는 .section에 tabindex가 없으면 -1을 넣는다. 메뉴로 이동한 뒤 그 절이 키보드 포커스를 받을 수 있게 한다. 이미 있는 tabindex는 건드리지 않는다.

시작점

document.readyState가 loading이면 DOMContentLoaded에서, 아니면 바로 initSmoothNav, initResourceDropdown, initThemeToggle, initSectionTargets를 부른다.

이 파일이 부르는 다른 파일

js/main.js는 다른 스크립트를 부르지 않는다. index.html이 css/site-header.css, css/styles.css, js/main.js를 읽는다. 자료 메뉴의 바깥 페이지는 각 페이지의 HTML로 넘어간다.

이력서 기간

이 파일이 맡는 일

cv/resume.html은 경력, 학력, 자격증을 적은 문서형 페이지다. 기간 숫자는 HTML에 미리 적혀 있지 않다. cv/js/ui.js의 fillCareerPeriods가 각 경력 항목의 시작일과 종료일을 읽어 .periodResult에 괄호로 넣는다. 페이지는 jQuery의 $(document).ready에서 이 함수를 한 번 부른다. 이후 상태가 바뀌는 코드는 없다.

cv/css/reset.css는 브라우저 기본 여백을 지우는 리셋이다. cv/css/style.css는 문서 폭, 절 제목, 경력 항목의 줄을 정한다. 상단 헤더를 쓰는 페이지라면 공통 css/site-header.css와 테마 스크립트가 따로 연결된다. 기간 계산 자체는 테마와 무관하다.

경력 항목의 읽는 조각은 .article 하나다. 그 안에 .startDate, .endDate, .periodResult가 있다. 시작일 글자가 비어 있으면 그 항목은 건너뛴다.

읽는 순서

1. .article을 하나씩 돈다.
2. 시작일이 있을 때만 종료일을 고르고 기간을 계산한다.
3. 결과를 .periodResult에 쓴다.

구간 설명

fillCareerPeriods

인자는 없다. 되돌리는 값도 없다. 각 .article에서 .startDate 텍스트를 앞뒤 공백 없이 읽는다. 빈 문자열이면 그 항목은 그대로 둔다.

시작일이 있으면 .으로 나누고, 공백을 지운 뒤 빈 조각을 버린다. 종료일 글자가 today이면 오늘 날짜로 바꾼다.

```
if (endDate === 'today') {
    const now = new Date();
    endDateArr = [year, month, day];
    $(this).find('.endDate').text(`${year}. ${month}. ${day}.`);
}
```

today가 아니면 종료일 글자도 .으로 나누고, 월과 일을 두 자리로 맞춘 뒤 같은 형식으로 .endDate에 다시 쓴다. 화면의 today 문구는 이 시점에 구체적인 날짜로 바뀐다.

계산은 `new Date(연, 월, 일)` 두 개의 차이로 한다. 월 인자는 나눈 문자열의 두 번째 조각을 그대로 넘긴다. 자바스크립트 `Date`의 월은 0부터 세므로, 달력의 1월이 두 번째 인자 `1`로 들어가면 2월로 잡힌다. 일수는 그 밀리초 차이를 하루로 나눈 값이다.

연은 일수를 365로 나눈 몫이다. 남은 일을 30으로 나누어 반올림한 값이 개월이다. 둘 다 0이면 글자는 `-`다. 연만 0이면 `N개월`, 개월만 0이면 `N년`, 둘 다 있으면 `N년 N개월`이다. 이 글자를 괄호로 감싸 `.periodResult`에 넣는다.

시작일이 없는 항목, 또는 `.periodResult`가 없는 항목은 기간 칸이 생기지 않는다. 종료일 조각이 모자라면 `endDateArr`를 읽을 때 그 항목에서 예외가 날 수 있다. 함수는 그 예외를 잡지 않는다.

이 파일이 부르는 다른 파일

`cv/js/ui.js`는 jQuery의 `$`만 사용한다. `cv/resume.html`이 jQuery와 이 파일을 읽는다. 다른 페이지의 스크립트를 부르지 않는다.

자기소개 페이지

이 파일이 맡는 일

cover_letter/profile.html은 프로필, 관심 분야, 학습 경험, 지원 동기를 절으로 나눈 페이지다. cover_letter/js/script.js는 지금 보이는 절에 맞춰 사이드 메뉴를 표시하고, 맨 위로 버튼을 연결한다. 즉시 실행 함수이고, DOM이 준비되면 initScrollSpy와 initToTop만 부른다.

cover_letter/css/styles.css는 본문 폭, 사이드 메뉴, 활성 링크, 맨 위로 버튼의 위치를 정한다. 공통 헤더가 있으면 테마와 모바일 메뉴는 js/main.js와 css/site-header.css가 맡는다. 이 파일은 테마 저장소를 건드리지 않는다.

HTML에서 이 스크립트가 찾는 조각은 이렇다.

- .content-section[id]: 관찰 대상 절
- .nav-link[data-section]: 절 id와 같은 값이면 활성
- #toTop: 맨 위로 버튼

읽는 순서

1. setActiveNav
2. initScrollSpy
3. initToTop
4. DOMContentLoaded에서 둘을 호출

구간 설명

setActiveNav

인자 id는 현재 절의 id다. 되돌리는 값은 없다. 모든 .nav-link를 돌며 data-section이 id와 같으면 is-active를 붙이고, 아니면 뺀다. 한 번에 여러 링크가 같은 data-section을 가지면 그 링크들은 함께 활성이 된다.

initScrollSpy

.content-section[id]가 없으면 바로 돌아간다. 있으면 IntersectionObserver를 만든다. 루트는 브라우저 뷰포트다. rootMargin은 -20% 0px -55% 0px라서 화면 가운데에 가까운 띠만 본다. threshold는 0, 0.25, 0.5, 0.75, 1이다.

항목이 교차하고 `intersectionRatio`가 0.25 이상일 때만 `setActiveNav(entry.target.id)`를 부른다. 비율이 그보다 작으면 활성 메뉴를 바꾸지 않는다. 스크롤로 절이 그 띠를 벗어나도 이전 활성 클래스를 이 콜백이 직접 지우지는 않는다. 다른 절이 조건을 만족할 때 `setActiveNav`가 나머지를 지운다.

관찰을 시작하기 전에 첫 절 id로 `setActiveNav`를 한 번 부른다. 그래서 로드 직후 메뉴는 첫 절을 가리킨다.

initToTop

`#toTop`이 없으면 돌아간다. 클릭하면 `window.scrollTo({ top: 0, behavior: "smooth" })`만 한다. 동작 줄이기 설정을 보지 않으므로, 줄이기가 켜져 있어도 이 버튼은 부드러운 스크롤을 요청한다.

이 파일이 부르는 다른 파일

다른 스크립트를 임포트하지 않는다. `cover_letter/profile.html`이 CSS와 이 파일을 읽는다.

온라인 명함

이 파일이 맡는 일

`online_business_card/business-card.html`은 긴 스크롤 구간 위에 명함을 고정해 두고, 스크롤에 따라 카드를 뒤집고 확대한다. 동작은 `online_business_card/js/script.js` 한 파일이다. `DOMContentLoaded`에서 카드, 스크롤 컨테이너, 이력 절, 스크롤 안내를 찾고 스크롤 리스너와 관찰자를 붙인다.

`online_business_card/css/style.css`는 `.scroll-container`의 높이와 카드의 `position: sticky`, 앞면·뒷면, `.resume-content section`의 숨김과 `.visible` 표시를 정한다. 스크립트는 transform과 opacity, `is-hidden`, `visible`만 바꾸고 레이아웃 숫자의 대부분은 CSS에 있다.

HTML에서 이 스크립트가 찾는 조각은 이렇다.

- `#card`: transform을 받는 명함
- `.scroll-container`: 진행률의 기준 높이
- `.resume-content section`: 20% 이상 보이면 `visible`
- `#scrollHint`: 24px보다 많이 내리면 `is-hidden`

읽는 순서

1. 요소를 찾고 `ticking`을 거짓으로 둔다.
2. `updateAnimation`을 정의하고 스크롤마다 `requestAnimationFrame`으로 한 번만 부른다.
3. 로드 직후 `updateAnimation`을 한 번 실행한다.
4. 이력 절에 `IntersectionObserver`를 붙인다.

구간 설명

updateAnimation

인자는 없다. `scrollY / (scrollContainer 높이 - 뷰포트 높이)`를 0과 1 사이로 자른 값이 `progress`다. 컨테이너가 뷰포트보다 작으면 분모가 0 이하가 되어 진행률이 숫자가 아닐 수 있다. 그 경우 뒤의 비교는 거짓 쪽으로 떨어져 앞 단계 분기로 가지 않는다.

진행률 0 이상 0.5 이하는 뒤집기다. `flipProgress`는 `progress / 0.5`다. `rotateX`는 10도에서 0도로 줄고, `rotateY`는 -15도에서 180도 쪽으로 195도를 곱해 더한다. `scale`은 1이고 opacity는 1이다.

0.5를 넘으면 확대다. `rotateX`는 0, `rotateY`는 180으로 고정한다. `scaleProgress`를 세 번 곱한 `easeScale`로 점점 가속하고, `scale`은 `1 + 40 * easeScale`이다. 끝에서는 약 41배다.

```
if (progress <= 0.5) {
    const flipProgress = progress / 0.5;
```

```
    rotateY = -15 + (195 * flipProgress);
    scale = 1;
} else {
    const easeScale = scaleProgress * scaleProgress * scaleProgress;
    scale = 1 + (40 * easeScale);
}
```

계산이 끝나면 `#card`의 `transform`에 `rotateX`, `rotateY`, `scale`을 한 줄로 넣는다. `#scrollHint`가 있으면 `scrollY > 24`일 때 `is-hidden`을 붙인다. 맨 아래 `ticking`을 거짓으로 되돌려 다음 스크롤이 다시 프레임을 요청할 수 있게 한다.

스크롤 리스너

스크롤마다 `ticking`이 거짓일 때만 `requestAnimationFrame(updateAnimation)`을 요청하고 `ticking`을 참으로 둔다. 한 프레임 안에 스크롤이 여러 번 와도 `updateAnimation`은 한 번만 돈다. 로드 직후 한 번 호출하므로 새로고침 때의 스크롤 위치도 반영된다.

이력 절 관찰

`threshold` 0.2다. 절이 뷰포트의 20% 이상 교차하면 `visible`을 붙인다. 교차가 끝나면 `visible`을 뺀다. 한 번 나타난 뒤 관찰을 끊지 않으므로, 위로 올리면 절이 다시 숨는 클래스 상태로 돌아간다. 관찰자 콜백의 두 번째 인자는 쓰이지 않는다.

이 파일이 부르는 다른 파일

다른 JS 모듈을 부르지 않는다. `business-card.html`이 CSS와 이 스크립트를 읽는다. 공통 헤더를 포함했다면 테마는 그 페이지의 테마 스크립트가 따로 처리한다.

학습 기록 대시보드

이 파일이 맡는 일

study_note_record_organization/dashboard/index.html은 히어로, 소개, 검색, 카드 그리드, 요약 모달, 원본 보기를 담는다. 카드와 모달의 동작은 dashboard/js/app.js다. 히어로 전환은 hero.js가 맡으며 다음 장에서 읽는다. 스타일은 dashboard/css/styles.css가 카드, 모달, 빈 상태, 히어로 겹침을 정한다.

페이지가 읽는 데이터는 빌드가 만든 js/data.js의 window.SITE_DATA다. 필드가 쓰이는 곳은 이렇다.

- catalog: 카드 배열. type, title, slug, summary, tags
- articles: 슬러그에서 요약 HTML로
- sourceMap: 원본 키에서 js/sources/ JSON 파일 이름으로
- sourceBuildId: 원본 요청의 v 쿼리
- sources: 옛 형식. sourceMap이 없을 때만 캐시로 복사한다

HTML 조각은 #subtitle, #info-compose, #note, #footer-count, #search, #grid, #empty, #modal-overlay, #modal-meta, #modal-body, #modal-close, #source-overlay, #source-title, #source-body, #source-close다.

파일 안의 상태는 catalog, articles, sourceMap, sourceCache, sourceBuildId, searchQuery다. 검색어만 입력할 때마다 바뀌고, 나머지 데이터는 init에서 한 번 채워진다.

읽는 순서

선언과 DOM 조회 다음에 함수가 나오고, 파일 끝에서 닫기·검색·init을 연결한다.

1. escapeHtml, getData, renderIntro, filteredItems, renderGrid
2. isAssignmentSourceKey, maskAssignmentPrivacy, htmlToPlainText, getSourceExtraImage, renderPlainSource
3. resolveSourceChunk, fetchSourceText, sourceDisplayName, openSourceViewer, closeSourceModal
4. bindSourceLinks, openModal, closeModal, init
5. 버튼, 배경, Escape, 검색 입력, DOMContentLoaded

구간 설명

escapeHtml, getData

escapeHtml은 임시 div의 textContent에 넣은 뒤 innerHTML을 돌려준다. null과 undefined는 빈 문자열이다. 카드 제목처럼 사용자 글을 HTML에 넣을 때 쓴다. 요약 본문 HTML은 빌드 결과이므로 이 함수를 거치지 않고 innerHTML에 들어간다.

getData는 window.SITE_DATA가 없으면 null이다.

renderIntro

카탈로그를 돌며 type이 study와 assignment인 개수를 센다. 그 외 종류는 총계에만 들어간다. #subtitle에는 고정 소개 문구를 넣는다. #info-compose에는 학습 건수와 과제 건수를 HTML로 넣는다. 숫자는 카탈로그에서 세므로 태그가 되지 않는다. #note에는 총개수와 두 빌드 명령을 적는다. #footer-count는 총 N개 기록이다. 요소가 없으면 그 칸만 건너뛴다.

filteredItems, renderGrid

filteredItems는 searchQuery의 앞뒤 공백을 지우고 소문자로 만든다. 비어 있으면 카탈로그 전체를 돌려준다. 있으면 제목, 요약, 종류, 태그를 한 문자열로 붙여 그 안에 검색어가 있는 항목만 남긴다. 태그가 없으면 빈 배열로 둔다.

renderGrid는 #grid를 비운다. 결과가 없으면 #empty의 hidden을 제거하고 돌아간다. 있으면 빈 문구를 숨기고, 항목마다 button.card.card--종류를 만든다. 라벨은 study면 STUDY NOTE, assignment면 ASSIGNMENT, 그 외는 RECORD다. 제목과 요약은 escapeHtml을 거친다. 요약이 없으면 요약 없음이다. 클릭하면 그 항목으로 openModal을 부른다.

원본 가리기와 평문

isAssignmentSourceKey는 경로에 과제분류하기 또는 data_과제분류하기가 있으면 참이다.

maskAssignmentPrivacy는 문자열을 돌려준다. 지도교수 줄은 OOO 교수님으로, 학번과 수강번호 숫자는 OOOOOOOO로 바꾼다. 괄호 안 7~8자리와 밑줄로 감싼 7~8자리도 같은 마스크다. 이 함수는 화면에서 원본을 열 때 한 번 더 적용된다. 빌드가 이미 가렸어도 같은 규칙을 다시 통과한다.

htmlToPlainText는 DOMParser로 HTML을 파싱해 body.innerText를 돌려준다. 본문이 없으면 입력 문자열을 그대로 둔다.

getSourceExtraImage는 키를 한 번 디코드한 뒤, 데이터베이스와 총괄이 같이 있으면 ER 그림 정보 { src, alt, caption }을 돌려준다. src는 assets/database-er-diagram.png다. 아니면 null이다. 디코드가 실패하면 원래 키로 검사한다.

renderPlainSource는 원문의 줄바꿈을 \n으로 맞추고 escapeHtml한 뒤 .source-plain HTML을 만든다. 추가 그림이 있으면 figure를 붙인다.

원본 불러오기

resolveSourceChunk는 sourceMap[key]를 먼저 본다. 없으면 키를 디코드해 한 번 더 본다. 둘 다 없으면 null이다.

fetchSourceText는 sourceCache[key]가 있으면 그 문자열로 바로 이행하는 프로미스를 돌려준다. 청크 이름이 없으면 null로 이행한다. 있으면 js/sources/파일이름을 현재 페이지 기준으로 만들고, sourceBuildId가 있으면 v 쿼리를 붙인다. cache: "no-store"로 fetch한다. 응답이 실패하면 거절되어 catch가 null을 돌려준다. 성공하면 payload.text를 캐시에 넣고, payload.key가 요청 키와 다르면 그 키에도 같은 텍스트를 넣는다.

sourceDisplayName은 링크 문구가 있으면 그것을, 없으면 경로의 마지막 조각을 쓴다. 끝의 (텍스트 추출본)은 뺀다. 결과가 비면 원본이다.

openSourceViewer, closeSourceModal

openSourceViewer(key, label)은 원본 오버레이의 hidden을 빼고 aria-hidden을 거짓으로 둔다. body의 overflow를 hidden으로 잠근다. 요약 모달이 아직 보이면 그 오버레이의 pointerEvents를 none으로 두어 원본이 위에 있는 동안 요약 쪽 클릭을 막는다.

제목을 넣고 본문은 원본을 불러오는 중…으로 바꾼 뒤 fetchSourceText를 기다린다. 텍스트가 null이면 build_static_site.py를 실행하라는 오류를 보여 준다. 키가 .html 또는 .htm으로 끝나면 htmlToPlainText를 먼저 적용한다. 과제 경로면 maskAssignmentPrivacy를 적용한다. 그 다음 renderPlainSource 결과로 본문을 바꾼다.

closeSourceModal은 원본 오버레이를 숨기고 pointerEvents를 비운다. 요약 모달도 숨겨져 있을 때만 body 스크롤 잠금을 푼다. 요약이 남아 있으면 잠금은 유지된다.

bindSourceLinks, openModal, closeModal

bindSourceLinks는 컨테이너 클릭에서 a[data-source] 또는 a.source-link만 가로챈다. 기본 이동과 버블을 막고, data-source가 있으면 openSourceViewer를 부른다. 키가 없는 링크는 이동만 막는다.

openModal(item)은 요약 오버레이를 열고 스크롤을 잠근다. 메타에는 라벨, 제목, 요약을 이스케이프해 넣는다. articles[item.slug]가 없으면 본문에 요약본이 없습니다.를 넣고 원본 링크는 연결하지 않는다. 있으면 .md-content로 넣고 bindSourceLinks를 부른다.

closeModal은 요약 오버레이를 숨긴다. 원본 오버레이가 숨겨져 있을 때만 스크롤 잠금을 푼다.

init과 이벤트

init은 SITE_DATA.catalog가 없으면 그리드에 data.js가 없습니다를 넣고 돌아간다. 있으면 카탈로그, 요약 HTML, 원본 지도, 빌드 아이디를 채우고 캐시를 비운다. sourceMap 없이 sources만 있으면 그 객체를 캐시로 쓴다. 이어서 renderIntro와 renderGrid를 부른다.

닫기 버튼은 각 모달을 닫는다. 오버레이 배경을 직접 클릭했을 때만 닫고, 안쪽 클릭은 유지한다. Escape는 원본이 열려 있으면 원본만 닫고, 아니면 요약이 열려 있을 때 요약을 닫는다. 검색 입력은 searchQuery를 바꾸고 renderGrid만 다시 부른다. 소개 문구는 다시 그리지 않는다.

readyState가 loading이면 DOMContentLoaded에서 init하고, 아니면 바로 한다.

이 파일이 부르는 다른 파일

window.SITE_DATA는 js/data.js가 만든다. 원본 본문은 js/sources/*.json을 fetch한다. 히어로 목록은 읽지 않는다. 빌드 스크립트는 실행하지 않고, 데이터가 없을 때 명령 이름만 화면에 적는다.

히어로 갤러리

이 파일이 맡는 일

dashboard/js/hero.js는 대시보드 상단의 사진 두 장을 겹쳐 두고 번갈아 보여 준다. 즉시 실행 함수이고, DOM이 준비되면 initHero만 부른다. 카드 목록과는 상태를 공유하지 않는다.

상수는 AUTO_MS 5500과 FADE_MS 1200이다. FADE_MS는 이 파일 안에서 타이머로 쓰이지 않는다. 사라지는 시간은 CSS 전환이 맡는다.

HTML 조각은 #hero, #hero-stage, #hero-img-a, #hero-img-b다. 두 이미지가 같은 자리에 겹치고, .is-visible인 쪽만 보인다.

읽는 순서

1. heroSrc, getHeroImages
2. initHero 안에서 visibleImg, hiddenImg, pickIndex, applyImage, swapTo, nextRandom, startTimer
3. 첫 장을 고르고 클릭과 타이머를 연결한다

구간 설명

heroSrc, getHeroImages

heroSrc(rel)은 data/image 기준 상대 경로에서 마지막 슬래시 뒤 파일 이름만 encodeURIComponent한다. 슬래시가 없으면 경로 전체를 encodeURI한다. 디렉터리 구분 슬래시는 인코딩하지 않는다.

getHeroImages는 window.HERO_IMAGES가 비어 있지 않으면 그것을 돌려준다. 없으면 window.SITE_DATA.heroImages를 본다. 둘 다 없으면 빈 배열이다. 지금 빌드는 hero-images.js에 window.HERO_IMAGES를 넣으므로 보통 첫 갈래를 탄다.

initHero

네 요소 중 하나라도 없으면 돌아간다. 이미지 목록이 비면 #hero에 hidden 클래스를 넣고 끝난다. 목록이 있으면 hidden 클래스와 hidden 속성을 제거한다.

안의 상태는 current 인덱스, 지금 A가 보이는지(showA), 전환 중인지(busy), timer다.

visibleImg와 hiddenImg는 showA에 따라 #hero-img-a와 #hero-img-b 중 하나를 돌려준다.

pickIndex는 목록이 한 장이면 0이다. 두 장 이상이면 current와 다른 인덱스가 나올 때까지 무작위로 고른다.

applyImage는 그 인덱스의 경로를 heroSrc로 바꿔 src에 넣고, alt는 학습·과제 기록 갤러리 이미지로 고정한다.

swapTo(index)는 busy이거나 index가 current와 같으면 아무 것도 하지 않는다. 아니면 busy를 참으로 두고, 숨은 이미지에 applyImage를 한 뒤 current를 먼저 바꾼다. 그 이미지가 이미 로드되어 있으면 바로 finish한다. 아니면 onload에서 finish하고, onerror에서는 busy만 거짓으로 되돌린다. 실패하면 보이던 장은 유지되고 current만 새 인덱스로 남아, 다음 무작위가 그 인덱스를 피할 수 있다.

finish는 새 이미지에 is-visible을 붙이고 이전 이미지에서 뺀다. showA를 새 이미지가 A인지로 맞추고 busy를 거짓으로 둔다.

nextRandom은 pickIndex 결과로 swapTo를 부른다.

startTimer는 기존 인터벌을 지운다. prefers-reduced-motion: reduce이거나 이미지가 한 장이면 새 인터벌을 만들지 않는다. 그 외에는 5500밀리초마다 nextRandom을 부른다.

초기화의 마지막은 첫 pickIndex로 A에 그림을 넣고 is-visible을 붙이는 것이다. 스테이지 클릭은 nextRandom 뒤 startTimer를 다시 불러 자동 전환 시간을 처음부터 센다. 동작 줄이기가 켜져 있으면 클릭으로만 넘긴다.

이 파일이 부르는 다른 파일

hero-images.js의 window.HERO_IMAGES 또는 data.js의 heroImages를 읽는다. app.js의 함수는 부르지 않는다. dashboard/index.html이 두 스크립트를 모두 읽는다.

요약 빌드

이 파일이 맡는 일

study_note_record_organization/scripts/build_output.py는 data/의 노트와 과제 문서를 읽어 output/에 요약 마크다운, catalog.json, index.md, 추출 텍스트를 쓴다. 끝난 뒤 build_static_site.py를 한 번 실행한다. extract_sources.py는 이 main만 부른다.

과제 프로그램의 VB 원문을 이 코드북에서 풀지는 않는다. 이 스크립트가 그 파일을 텍스트로 읽어 캐시에 넣는 경로만 적는다. 학번과 이름은 마스크 문자로만 설명한다.

CatalogItem 하나가 카드 하나다. SourceRef는 그 카드가 가리키는 원본 파일 하나다.

읽는 순서

1. 본문 추출과 비밀·개인정보 가리기
2. 링크 주소와 추출 캐시
3. 학습 단위 이름과 정렬
4. SourceRef, CatalogItem.load_sources
5. 학습 그룹과 학습 카탈로그
6. 과제 카탈로그와 과제 요약 문장
7. render_markdown, main

구간 설명

본문 추출

read_text_file은 경로의 텍스트를 읽는다. extract_docx, extract_pdf, extract_html은 각각 워드, PDF, HTML에서 글자만 뽑는다. HTML은 MLStripper가 태그를 버리고 텍스트를 모은다.

extract_source는 확장자를 본다. .txt와 .vb는 텍스트 파일, .docx, .pdf, .html, .htm은 위 추출기다. 다른 확장자는 빈 문자열이다. 결과는 항상 mask_secrets를 거친다.

mask_secrets는 본문이 비면 그대로 돌려준다. 아니면 고정된 키·토큰 정규식과, api_key처럼 이름 있는 대입의 값만 마스크로 바꾼다. 대입 패턴은 이름과 따옴표는 남기고 가운데 값만 가린다.

is_assignment_path는 경로에 과제분류하기가 있으면 참이다. mask_assignment_privacy는 지도교수 이름을 000 교수님으로, 학번·수강번호와 파일명 속 7~8자리 숫자를 00000000로 바꾼다. 빈 문자열은 그대로다.

링크와 추출 캐시

source_href는 data/를 떼고 경로 조각을 URL 인코딩해 /data/...를 만든다. extracted_href는 경로에서 영숫자와 밑줄, 하이픈만 남기고 120자로 잘라 /_extracted/이름.txt를 만든다.

source_link_target은 SourceRef의 종류로 보이는 이름과 주소를 정한다. 경로가 폴더이고 그 안에 Form1.vb가 있으면 그 VB로 연결한다. 워드와 PDF는 추출본 주소와 (텍스트 추출본) 꼬리를 쓴다. 텍스트, HTML, VB는 원본 /data/ 주소다. 그 외도 원본 주소다.

cache_extract는 같은 안전 이름으로 output/_extracted에 텍스트를 쓴다. 내용이 비면 [empty]를 쓴다. 쓰기 전에 mask_secrets를 한 번 더 거친다.

학습 단위

parse_unit은 파일 이름에서 장, 일차, 강, 주차, 서론, 프롤로그, 부록, 코딩면허를 찾는다. 없으면 전체다.

sort_key_unit은 정렬 튜플을 만든다. 전체가 먼저, 서론·프롤로그, 구간 숫자, 본문 숫자, 부록 순이다. 숫자가 없으면 99로 둔다.

SourceRef, CatalogItem.load_sources

SourceRef 필드는 path, unit, text다. text는 처음엔 빈 문자열이다.

CatalogItem 필드는 id, type, category, title, slug, summary, tags, sources다. load_sources는 각 출처를 돈다. 파일이면 extract_source로 text를 채운다. 폴더면 obj와 bin을 빼고 Form1.vb와 Form1.Designer.vb만 이어 붙인다. 과제 경로면 캐시에 넣기 전에 mask_assignment_privacy를 적용한다. 학습 경로는 추출 본문을 그대로 캐시한다. 카드의 text 필드 자체는 과제라도 이 함수에서 개인정보 마스크를 다시 씌우지 않는다. 마크다운을 만들 때 경로와 링크 문구를 따로 가린다.

학습 카탈로그

normalize_study_title은 확장자와 장·일차·독서노트 꼬리, 코딩면허, 실습 필기 꼬리를 떼고 과목 이름만 남긴다.

collect_study_groups는 data/ 바로 아래 파일만 본다. .hwpx는 건너뛰고 이름만 따로 모은다. .txt와 .docx만 과목 이름 아래로 모은다. STUDY_MERGE_ALIASES에 있으면 별칭을 같은 과목 키로 합친다. 각 묶음은 sort_key_unit으로 정렬한다. 그룹 딕셔너리와 건너뛴 hwpx 목록을 돌려준다.

build_study_catalog는 그룹 수가 37이 아니면 output/study-audit.txt에 그룹 이름과 hwpx 목록을 쓰고 SystemExit로 중단한다. 37이면 제목순으로 CatalogItem을 만든다. STUDY_META에 있으면 슬러그, 한 줄 요약, 태그, 분류를 그대로 쓴다. 없으면 제목으로 슬러그를 만들고 요약은 제목 학습 노트, 태그는 study, 분류는 제목에 Do it이 있으면 book 아니면 course다. id는 study-슬러그다. 출처의 unit은 parse_unit이다.

STUDY_NARRATIVES는 슬러그별 고정 서술이다. build_study_narrative는 그 문장과 원문에서 뽑은 단계 문단을 묶어 overview, main, reflection, units를 돌려준다. 원문이 짧으면 weave_narrative와 soften_sentence가 문장을 다듬고, 점이 없으면 안내 문장으로 채운다.

과제 카탈로그

find_assign_file은 아직 used에 없는 파일 중 제외 문자열이 없고 키워드가 들어 있는 첫 파일을 돌려준다. 키워드 순서로 찾고, 없으면 None이다.

build_assignment_catalog는 과제 폴더의 파일만 목록으로 삼는다. 안쪽 add는 키워드로 파일 하나를 잡아 assignment-슬러그 항목을 추가한다. 맞은 파일의 출처 단위는 총괄과제다. 맞는 파일이 없어도 항목은 만들어지고 출처만 빈다. 윈도우 프로그래밍1은 add를 쓰지 않고 출처를 빈 리스트로 직접 넣는다. 요약 문구는 WinForms 실행 프로그램 제출 (텍스트 문서 없음)이다. 나머지 과목은 과목 이름과 키워드 표로 add 한다. 인터넷프로그래밍은 이름에 프로그래밍2가 들어간 파일을 제외한다.

extract_meta_assignment는 학번, 수강번호, 이름, 지도교수, 제출일을 딕셔너리로 뽑는다. 이 값은 요약 문장을 만들 때 표지 정보로 쓰이며, 나중에 가리기 규칙을 탄다.

과제 본문 정리는 한 묶음이다.

- strip_assignment_cover_block, strip_assignment_meta_text, is_assignment_meta_bullet는 표지의 학번·교수·제출일 줄을 본문 요약에서 뺀다.
- summarize_assignment_bullets, extract_assignment_points는 남은 줄에서 항목을 고른다.
- _split_assignment_sections, _has_assignment_section_markers, _extract_assignment_topics는 절 제목이 있으면 절별로 나눈다.
- _sentence_from_section, _compose_summary_from_topics, _sentence_from_points, _assignment_summary_filler는 그 조각으로 문장을 만든다. 쓸 문장이 없으면 과목 이름에 기대는 채움 문장을 쓴다.
- build_assignment_content_summary는 위 결과를 문단 리스트로 돌려준다. render_markdown의 과제 본문이 이 리스트를 쓴다.

render_markdown

슬러그가 windows-programming-1이면 render_windows_programming1_markdown으로 간다. 그 함수는 원문 없이 프론트매터와 화면 설명, 캡처 경로를 적는다. 출처 목록은 비어 있다.

다른 항목은 출처 text를 이어 붙인다. 학습이면 build_study_narrative의 개요, 배운 점, 단위별 문단을 쓴다. 과제가 아닌데 학습 서술이 없으면 글머리표 요약이다. 글머리표도 없으면 원본이 짧거나 비었다는 한 줄을 넣는다. 과제면 build_assignment_content_summary의 문단과 배운 점 / 적용 고정 문장을 넣는다.

맨 아래 원본 출처는 source_link_target의 링크다. 과제면 보이는 이름과 경로에 mask_assignment_privacy를 적용한다. 반환 문자열 전체에 mask_secrets를 한 번 더 적용한다.

main

output/study, output/assignment, output/_extracted를 만든다. build_study_catalog가 37이 아니면 여기서 중단되므로 과제 카탈로그까지 가지 않는다. 통과하면 build_assignment_catalog를 부르고, 모든 항목에 load_sources를 한다.

이번 슬러그에 없는 예전 .md는 두 출력 폴더에서 지운다. 각 항목의 render_markdown 결과를 study 또는 assignment 폴더의 슬러그.md로 쓴다. catalog.json에는 id, 종류, 분류, 제목, 슬러그, 요약, 태그, 출력 경로, 출처 경로와 단위를 넣는다. index.md는 학습과 과제 링크 목록이다.

그 다음 scripts/build_static_site.py를 현재 파이썬으로 실행한다. check=False라서 정적 빌드가 실패해도 이 함수는 예외를 올리지 않는다. 스크립트 파일이 없거나 실행 자체가 OSError면 그 예외만 삼킨다.

이 파일이 부르는 다른 파일

data/의 노트와 과제 문서, 그리고 폴더 출처 안의 Form1.vb와 Form1.Designer.vb를 읽는다. 쓰는 곳은 output/이다. 마지막에 build_static_site.py를 subprocess로 부른다.

정적 사이트 데이터

이 파일이 맡는 일

study_note_record_organization/scripts/build_static_site.py는 output/catalog.json과 요약 마크다운을 읽어 대시보드가 바로 여는 파일을 만든다.

- dashboard/js/data.js의 window.SITE_DATA
- dashboard/js/sources/ 아래 원본 JSON
- dashboard/js/hero-images.js의 window.HERO_IMAGES
- dashboard/index.html의 data.js, app.js 쿼리

main은 build_site를 부르고, 브라우저로 dashboard/index.html을 열 수 있다고 안내한다.

읽는 순서

1. 가리기, 마크다운을 HTML로 바꾸기, 원본 링크 다시 쓰기
2. 원본 파일 읽기
3. 히어로 목록, 청크 이름, 빌드 아이디, 크기 한도
4. collect_catalog, build_site, main

구간 설명

가리기

mask_secrets와 mask_assignment_privacy, is_assignment_path는 build_output.py와 같은 역할이다. 이 파일에 따로 있다. 요약 HTML을 만들기 전과, 원본 JSON에 넣기 전에 한 번씩 탄다.

마크다운을 HTML로

parse_frontmatter는 앞의 --- 블록과 본문을 나눈다. 프론트매터는 카드에 다시 쓰지 않고 본문만 HTML로 간다. 카드 필드는 catalog.json에서 collect_catalog가 고른다.

md_to_html은 제목, 목록, 문단, 이미지를 HTML로 바꾼다. _esc는 글자를 이스케이프하고, _inline_md는 **굵게**, ` 코드 `, 슬래시로 시작하는 마크다운 링크를 인라인 HTML로 바꾼다. 상대 경로 이미지는 그대로 둔다.

normalize_source_key는 역슬래시를 슬래시로 바꾸고 앞의 슬래시를 뗀 뒤, 퍼센트 인코딩이 더 풀리지 않을 때까지 unquote한다.

rewrite_source_links는 HTML 안의 /data와 /_extracted 링크를 찾는다. 주소에서 원본 키를 꺼내 pending 집합에 넣고, href="#"와 data-source="키", source-link 클래스를 붙인다. 이미 class가 있으면 그 값 뒤에 source-link를 더한다. 대시보드의 bindSourceLinks가 이 앵커를 누를 때 원본 보기를 연다.

원본 읽기

resolve_source_path는 _extracted/로 시작하면 output/ 아래로, data/로 시작하면 저장소 루트 아래로 경로를 만든다. 둘 다 아니면 None이다.

read_text_file은 utf-8, utf-8-sig, cp949 순으로 읽는다. 모두 실패하면 깨진 글자를 대체해 utf-8로 읽는다.

read_source_text는 경로가 파일이 아니거나 읽기 OSError면 None이다. 읽은 뒤에는 mask_secrets를 적용하고, 과제 경로면 mask_assignment_privacy도 적용한다.

히어로, 청크, 한도

collect_hero_images는 data/image의 파일을 이름순으로 보고, 이미지 확장자만 ../data/image/파일이름으로 모은다. 폴더가 없으면 빈 리스트다.

write_hero_images_js는 그 목록을 window.HERO_IMAGES = ...;로 hero-images.js에 쓰고 개수를 돌려준다.

source_chunk_filename은 원본 키 해시 8자와 본문 해시 8자를 이어 키-본문.json 이름을 만든다. 본문이 바뀌면 파일 이름이 바뀐다.

source_build_id는 정렬한 키:파일이름 전체를 해시해 12자를 돌려준다. stamp_index_scripts는 dashboard/index.html에서 js/data.js와 js/app.js의 기존 쿼리를 지우고 ?v=빌드아이디를 붙인다. hero.js 주소는 바꾸지 않는다.

assert_under_github_limit은 파일 크기가 100MB를 넘으면 SystemExit로 빌드를 멈춘다. 청크마다, 그리고 data.js를 쓴 뒤에 호출한다.

collect_catalog, build_site

collect_catalog는 catalog.json의 items에서 id, type, title, slug, summary, tags만 남긴다. 요약이 없으면 빈 문자열, 태그가 없으면 빈 배열이다. 분류와 출처 목록은 카드 데이터에 넣지 않는다.

build_site는 catalog.json이 없으면 python scripts/build_output.py를 먼저 실행하라는 메시지로 중단한다.

항목마다 output 경로의 마크다운을 연다. 파일이 없으면 그 항목의 요약 HTML은 만들지 않는다. 있으면 본문만 mask_secrets한 뒤 md_to_html과 rewrite_source_links를 거쳐 articles[slug]에 넣는다. 링크가 모은 원본 키는 pending_sources에 쌓인다.

dashboard/js/sources가 있으면 폴더를 지우고 다시 만든다. 키를 정렬해 read_source_text가 None이 아닌 것만 JSON으로 쓴다. 객체는 key와 text다. 청크가 한도를 넘으면 그 시점에 중단한다. sourceMap은 원본 키에서 청크 파일 이름으로 간다.

SITE_DATA에는 catalog, articles, sourceMap, sourceBuildId, counts가 들어간다. counts는 catalog.json의 값을 그대로 둔다. data.js는 window.SITE_DATA = 뒤에 그 JSON을 붙인 한 파일이다. 한도를 확인한 뒤 stamp_index_scripts와 write_hero_images_js를 부른다.

main은 build_site가 예외 없이 끝나면 완료 문장을 출력한다. 한도 초과나 catalog.json 없음은 SystemExit로 올라간다.

이 파일이 부르는 다른 파일

output/catalog.json, output/study와 output/assignment의 마크다운, output/_extracted 또는 data/의 원본, data/image를 읽는다. dashboard/js/data.js, dashboard/js/sources/*.json, dashboard/js/hero-images.js, dashboard/index.html을 쓴다. app.js와 hero.js의 내용은 바꾸지 않는다.

문서 생성 스크립트

이 파일이 맡는 일

`docs/`의 파이썬 스크립트는 사이트를 그리지 않는다. 이미 있는 마크다운이나 소스 주석을 한 권의 HTML로 합친 뒤 Edge 또는 Chrome으로 PDF를 인쇄한다. 글꼴은 맑은 고딕이다.

다섯 갈래다.

- `build_portfolio_book.py`는 `docs/portfolio-book/*.md`를 합치고 Mermaid 다이어그램을 그려 `portfolio-book.md`, HTML, PDF 단행본으로 만든다.
- `build_code_reference.py`는 함수·클래스 주석과 유사코드를 `code-reference.md`, HTML, PDF로 만든다.
- `build_specs.py`는 `docs/specs/*.md`를 `technical-specs.md`, HTML, PDF로 만든다. Mermaid 차트는 그림으로 바꾼다.
- `build_codebook.py`는 `docs/codebook/*.md`를 `codebook.md`, HTML, PDF로 만든다.
- `build_source_bundle.py`는 화면과 빌드 소스를 파일 순서대로 이어 행 번호가 있는 `source-bundle.html`, PDF로 만든다.

`data/` 과제 프로그램 소스는 코드 설명 수집 대상이 아니다. 파일 목록에 학번이나 이름이 섞이면 마스크로 바꾼다.

읽는 순서

각 스크립트는 글을 모으고, 마크다운의 일부만 HTML로 바꾼 다음, 브라우저의 `print-to-pdf`를 호출하는 순서다.

구간 설명

build_code_reference.py

소스 트리를 돌며 자바스크립트 JSDoc, 파이썬 docstring, HTML·CSS 파일 머리 주석을 모은다. 생성물인 `data.js`, `hero-images.js`, `sources` JSON과 `data/` 아래 코드 확장자는 본문으로 풀지 않는다. 심볼마다 설명, 인자, 반환을 적고, 함수 본문에서 만든 유사코드를 붙인다. 클래스면 필드와 메서드를 적는다. 학번 숫자와 이름을 마스크한 뒤 마크다운으로 쓰고, 같은 HTML 변환으로 PDF를 만든다.

build_specs.py

`docs/specs`의 마크다운을 파일 이름순으로 잇는다. 제목, 목록, 코드 울타리, 문단, 굵게, 인라인 코드를 HTML로 바꾼다. 울타리 언어가 `mermaid`이면 차트 소스를 브라우저로 그려 PNG로 저장하고 `img`로 넣는다. 유스케이스 소스에 `actor-style: person`이 있으면 Mermaid 대신 사람 그림이 있는 SVG를 그린다.

나머지 차트는 Mermaid다. 그림 파일 이름은 소스 해시다. 이번 빌드에 쓰이지 않은 그림은 지운다. PDF 인쇄는 머리글과 바닥글 없이 한다.

build_codebook.py

`docs/codebook`의 마크다운을 파일 이름순으로 잇는다. HTML로 바꾸는 규칙은 명세와 같고, 차트 분기는 없다. 코드 울타리는 회색 `pre`로만 둔다. 브라우저를 찾지 못하거나 PDF가 너무 작으면 중단한다.

build_portfolio_book.py

`docs/portfolio-book`의 마크다운 챕터들을 순서대로 잇는다. 표지, 목차, 인쇄 스타일을 입히고 Mermaid 다이어그램을 브라우저 헤드리스 모드로 렌더링하여 이미지로 삽입한 뒤, A4 규격 단행본 `portfolio-book.pdf`를 인쇄한다.

build_source_bundle.py

HTML, CSS, JS, Python 소스를 찾아 각 줄마다 행 번호를 매긴 뒤 단일 HTML 및 PDF로 병합한다. 생성 파일과 라이브러리, 원본 데이터는 제외한다.

이 파일이 부르는 다른 파일

코드 설명 스크립트는 저장소의 화면 스크립트, 스타일, 빌드 스크립트를 읽는다. 명세와 코드북 스크립트는 `docs/specs`와 `docs/codebook`의 마크다운만 읽는다. 셋 다 Edge 또는 Chrome 실행 파일을 찾아 PDF를 쓴다.

리소스 보호 및 보안 가드

이 파일이 맡는 일

`js/protect-resources.js`는 정적 호스팅(GitHub Pages) 환경에서 무단 복사, 스크랩, 악의적인 iframe 끼워 넣기를 방지하고 방문자에게 안내를 제공하는 클라이언트 보안 스크립트다.

함께 동작하는 거버넌스 설정은 다음과 같다:

- `robots.txt`: 일반 검색엔진에는 메인 페이지만 색인하도록 하고, 하위 자료 폴더와 17종의 인공지능 학습 크롤러는 사이트 전체에서 거절한다.
- `.well-known/tdmrep.json`: TDM(Text and Data Mining) 권리 유보 선언을 머신러닝 수집기에 전달한다.

읽는 순서

1. 허용 호스트(`ALLOWED_HOSTS`) 및 상수 정의 (`MESSAGE`, `HIDE_MS`)
2. `isEmbedded`, `allowedHost`, `window.__portfolioGuard`
3. 차단 화면 그리기 (`paintBlockedPage`) 및 조건 검사
4. 안내 토스트 스타일 및 키프레임 주입 (`#resource-protect-notice`)
5. 입력 요소 판별 (`isEditable`), 안내 표시 (`showNotice`), 이벤트 차단 (`block`)
6. 마우스 이벤트 리스너 및 키보드 단축키 리스너 등록

구간 설명

isEmbedded

`window.top !== window.self`를 검사한다. 교차 출처 iframe에 갇혀 예외가 발생하더라도 `catch`에서 참을 돌려주어 임베딩 상태로 판단한다.

allowedHost

접속 프로토콜이 `http:` 또는 `https:`인지 확인하고, `window.location.hostname`이 `ackdark23358.github.io`, `localhost`, `127.0.0.1`에 해당하는지 검사한다.

paintBlockedPage

비인가 호스트나 iframe 환경일 때 실행된다. `document.body`의 내용을 모두 비우고 검은 배경에 "자료에 대해서 퍼가는 것을 방지하였습니다."라는 문구만 전체 화면 중앙에 띄운다. 본문 스크립트와 스타일 주입은 즉시 중단된다.

isEditable

이벤트가 발생한 대상 요소(event.target)가 검색창이나 텍스트 입력창(input, textarea, select, contenteditable) 내부인지 검사한다. 학습록 대시보드에서 방문자가 검색어를 타이핑하는 동작이 차단되지 않도록 보호한다.

showNotice와 block

copy, cut, contextmenu(우클릭), dragstart, selectstart 이벤트 및 Ctrl+C, Ctrl+X, Ctrl+A, Ctrl+S, Ctrl+U 단축키가 발생하면 e.preventDefault()로 가로채고 showNotice()를 호출한다. 하단에 토스트 팝업이 부드럽게 떠올랐다가 2.6초 뒤 사라진다.

이 파일이 부르는 다른 파일

외부 의존성은 없다. 5개 공개 HTML(index.html, profile.html, business-card.html, resume.html, dashboard/index.html)의 바닥에서 로드되어 독립적으로 동작한다.